

Cahier des charges — MVP

Application de gestion de syndic (Maroc)

Version : 1.4 (MVP) **Langue de l'interface :** Français uniquement **Cadre légal :** Loi 18-00 (copropriété) · Loi 09-08 (protection des données personnelles)

1. Contexte et objectifs

L'application vise à digitaliser la gestion d'un syndic de copropriété au Maroc. Elle s'adresse à un **syndic professionnel gérant plusieurs résidences** et permet de centraliser la gestion des copropriétaires, le suivi des paiements, les assemblées générales, la communication, la maintenance et les documents officiels.

Deux interfaces distinctes s'appuient sur un backend unique :

- un **Back Office (BO) web** pour les agents du syndic (gestion + export du travail) ;
- une **application mobile** pour les copropriétaires et locataires (consultation, paiement, réclamations).

L'objectif du MVP est de couvrir le cycle de gestion courant sans paiement en ligne automatisé (différé en v2), en validant manuellement les justificatifs de paiement. L'ensemble est conçu pour être **développé et hébergé gratuitement** (architecture monolithique légère sur une seule machine).

2. Périmètre du MVP

2.1 Modules inclus

1. Gestion des copropriétaires et de la structure (résidences / GH / immeubles / lots)
2. Paiement par upload de justificatif et validation manuelle
3. Assemblées générales (hors vote électronique)
4. Communication (push au lancement ; WhatsApp en v2), en français
5. Maintenance et signalements
6. Gestion documentaire (GED)

2.2 Exclut du MVP (versions ultérieures)

- Paiement en ligne CMI (le module paiement est conçu pour l'accueillir sans refonte)
- Vote électronique en AG (quorum, procurations, dépouillement)
- Comptabilité complète (budget prévisionnel, fonds de réserve, rapprochement bancaire)

- Multilingue arabe et support RTL
 - **Quittance / facture PDF** : remplacée par une **notification de confirmation** ; la preuve durable est l'historique des paiements dans l'app
 - **WhatsApp** : reporté en v2 (push uniquement au lancement, pour rester gratuit)
-

3. Acteurs et rôles

Acteur	Interface	Description
Agent syndic	BO web	Gère les résidences, valide les paiements, traite les signalements, organise les AG, publie les documents.
Copropriétaire / Résident	App mobile	Consulte son solde et ses lots, dépose des justificatifs de paiement, crée des réclamations/incidents, accède aux AG et documents.
Conseil syndical (optionnel v1)	BO web (lecture)	Accès en consultation aux rapports et décisions.

Le résident porte un **type** : `propriétaire` ou `locataire`.

4. Architecture générale

Architecture **monolithe modulaire .NET** : une seule API ASP.NET Core, un seul déployable, servant les deux clients.

Clients

- **BO web (agents)** : React.js (Vite)
- **App mobile (résidents)** : React Native (Expo) — push notifications

Backend (.NET 10 LTS / C# 14)

- Une **API ASP.NET Core unique**, organisée en **modules** (bounded contexts) : Identity, Copropriété, Paiement, Assemblées, Maintenance & Signalements, GED, Notification
- **Communication inter-modules** : en mémoire — interfaces injectées (synchrone) + événements internes **MediatR** (découplage). **Pas de gateway, pas de broker, pas de service discovery** (inutiles dans un seul processus).
- **Persistence** : une seule base **PostgreSQL** avec un `DbContext` **unique** (schéma unique) + EF Core
- **Auth** : ASP.NET Core Identity + **JWT**, isolation par **politiques d'autorisation** (rôle + appartenance)

- **Jobs de fond** : Hangfire / Quartz.NET / `BackgroundService` (purge, envoi de notifications)
- **Conteneurisation** : Docker + Docker Compose (**pas de Kubernetes au MVP**)
- **CI/CD** : GitHub Actions

Sécurité / isolation des données Toute l'autorisation est centralisée dans l'API : un résident n'accède qu'à ses lots, paiements et signalements ; un agent qu'à ses résidences.

Évolutivité : les modules ont des frontières nettes (interfaces + événements internes + schémas séparés). Si un module doit scaler seul plus tard, il peut être **extraît en service** sans repenser le domaine — on garde la porte ouverte sans en payer le prix maintenant.

5. Structure des données — hiérarchie

```

Résidence (ex. "Les Jardins")
├── Groupe d'Habitation (GH) ← regroupe en général 2 immeubles
│   ├── Immeuble (ex. "Bloc A")
│   │   ├── Lot (appartement / local commercial)
│   │   │   └── Résident ← un résident peut détenir plusieurs
│   │   │       lots

```

Règles de structure :

- Une résidence contient 1 à N **groupes d'habitation (GH)**.
- Un GH regroupe **2 immeubles** (modélisé en 1 à N pour rester flexible : un GH peut ne contenir qu'un seul immeuble, ce qui couvre le cas d'une résidence à immeuble unique).
- Un immeuble contient N lots.
- Un lot est de type `appartement` ou `local commercial`.
- Un résident peut être lié à plusieurs lots (table de liaison).
- Le **solde est porté par le lot** ; l'espace mobile affiche au résident le total agrégé de ses lots.

6. Spécifications fonctionnelles par module

6.1 Gestion des copropriétaires et de la structure

BO (agent) :

- Création / édition des résidences, groupes d'habitation, immeubles et lots.
- Annuaire des copropriétaires : coordonnées (nom, prénom, email, téléphone), type (propriétaire / locataire), statut (actif / inactif).

- Fiche par lot : numéro, type (appartement / local commercial), étage, immeuble de rattachement, résident(s) associé(s), solde courant.
- Rattachement d'un ou plusieurs lots à un résident.
- **Création du compte résident** : l'agent crée le résident puis envoie une **invitation** (lien ou code) permettant l'activation de l'app mobile.

App mobile (résident) :

- Espace personnel : consultation de ses lots, de son solde global et par lot.
- Consultation de ses coordonnées (modification soumise à validation de l'agent — optionnel v1).

6.2 Paiement (upload + validation manuelle)

Principe : le résident règle par un moyen externe (virement, chèque, espèces) puis dépose un justificatif. L'agent valide ; le solde du lot est alors déduit et un **message de confirmation** est envoyé au résident.

App mobile (résident) :

- Sélection du lot concerné (si plusieurs lots).
- Saisie d'un **montant libre**.
- Sélection de la **période** concernée (ex. mois ou trimestre).
- Upload d'un **justificatif** (image ou PDF).
- Soumission → statut « **En attente d'approbation** ».
- Suivi du statut et **historique des paiements** (qui fait office de preuve durable).

BO (agent) :

- File des paiements en attente de validation.
- Consultation du justificatif.
- **Validation** → déduction du montant sur le solde du lot + **notification de confirmation** au résident (montant, période, nouveau solde).
- **Rejet** → saisie d'un motif + notification au résident (possibilité de resoumettre).
- **Gestion du solde (charges)** : l'agent peut saisir un **ajustement de solde** (charge périodique, régularisation) pour *augmenter* le solde d'un lot. *Ce mécanisme remplace, pour le MVP, le module d'appels de fonds : sans lui, le solde ne pourrait que diminuer.*

Confirmation : pas de quittance PDF au MVP. La validation déclenche une **notification de confirmation** ; la **preuve durable** est l'**historique des paiements validés** consultable dans l'app (date, montant, période).

Statuts d'un paiement : en_attente → valide / rejete (annule optionnel).

6.3 Assemblées générales (AG)

Périmètre v1 (hors vote électronique) :

- BO : création d'une AG (titre, date, lieu, ordre du jour).
- BO : génération et envoi des **convocations** (via le module communication).
- BO : émargement / feuille de présence numérique (optionnel v1).
- BO : upload du **procès-verbal (PV)** après la séance ; le PV est automatiquement archivé dans la GED.
- App mobile : consultation des convocations, de l'ordre du jour et des PV.

Reporté en v2 : vote électronique, gestion du quorum et des procurations, dépouillement.

6.4 Communication

- **Canaux :** notifications **push** (Expo / FCM-APNs) — **seul canal au lancement**. **WhatsApp** (Cloud API) reporté en v2.
- **Langue :** français uniquement.
- **Couche unifiée :** un service interne de notification choisit le(s) canal(aux) selon les préférences du résident, puis journalise l'envoi. Architecturé multi-canal pour accueillir WhatsApp en v2 sans refonte.
- **Préférences par résident :** activation/désactivation du push (WhatsApp en v2) — consentement, conformité loi 09-08.
- **Événements déclencheurs :** confirmation/rejet de paiement, convocation AG, publication d'un PV, mise à jour d'un signalement, annonce de maintenance, annonces générales.
- **Journal des envois :** canal, type d'événement, statut (envoyé / livré / échec), horodatage.

Pour la v2 (WhatsApp) : tout message à l'initiative du syndic (hors fenêtre de 24 h) nécessitera un **template pré-approuvé par Meta**. À anticiper le moment venu.

6.5 Maintenance et signalements

Signalements (table unifiée) : un seul flux pour deux types — **réclamation** (résident) et **incident technique**.

App mobile (résident) :

- Créer un signalement : type, titre, description, photo (optionnelle), lot/immeuble concerné.
- Suivre le statut et échanger.

BO (agent) :

- File des signalements, assignation, réponse, changement de statut.

- Statuts : `reçu` → `en_cours` → `résolu` → `clôturé`.

Maintenance préventive planifiée :

- BO : planification d'actions (ascenseur, pompes, nettoyage, etc.) avec date prévue et récurrence éventuelle.
- Affichage sous forme de **widgets** « **actions à venir** » (BO et/ou résident) ou de **annonces** diffusées aux résidents.
- Statuts : `à_venir` → `en_cours` → `terminée`.

6.6 Gestion documentaire (GED)

- Documents gérés au MVP** : règlement de copropriété, PV d'AG archivés.
- BO : upload, catégorisation (type de document), définition de la visibilité (résidents oui/non), rattachement à une résidence/immeuble.
- App mobile : consultation et téléchargement des documents autorisés.
- Stockage : système de fichiers de la VM** (volume Docker dédié) + **métadonnées en base**. Le fichier n'est jamais stocké en base.
- Conservation** : les documents GED (règlement, PV d'AG) sont **conservés indéfiniment** (documents légaux), **jamais purgés**.

7. Modèle de données — tables principales

Table	Champs clés
<code>residences</code>	id, nom, adresse, ville
<code>groupes_habitation</code>	id, residence_id, nom
<code>immeubles</code>	id, groupe_habitation_id, nom_bloc, adresse, nb_etages
<code>lots</code>	id, immeuble_id, numero, type (<code>appartement</code> / <code>local_commercial</code>), etage, superficie, solde
<code>residents</code>	id, nom, prenom, email, telephone, type (<code>proprietaire</code> / <code>locataire</code>), statut (<code>actif</code> / <code>inactif</code>), compte_active
<code>lot_resident</code>	id, lot_id, resident_id, type, date_debut, date_fin
<code>paiements</code>	id, lot_id, resident_id, montant, periode, mode_paiement, justificatif_path, statut, valide_par, date_validation, motif_rejet, created_at

Table	Champs clés
ajustements_solde	id, lot_id, montant, type (charge)(regularisation), libelle, periode, cree_par, created_at
assemblees	id, residence_id, titre, date, lieu, ordre_du_jour, statut, pv_document_id
signalements	id, type (reclamation)(incident), lot_id, immeuble_id, resident_id, titre, description, photo_path, statut, assigne_a, reponse, created_at, updated_at
maintenance_planifiee	id, residence_id, immeuble_id, type, libelle, date_prevue, recurrence, statut, visible_residents
documents	id, residence_id, immeuble_id, type (reglement)(pv_ag)(autre), titre, fichier_path, date, visible_residents
preferences_notification	id, resident_id, canal_push (canal_whatsapp en v2)
devices	id, resident_id, push_token, plateforme, derniere_activite
notifications_log	id, resident_id, canal, type_evenement, statut, created_at

Organisation en modules (monolithe modulaire, base unique, DbContext unique) :

Toutes les tables vivent dans **une seule base PostgreSQL**, sous un **schéma unique** et un **DbContext unique**. Les tables sont regroupées **logiquement** par module (organisation du code uniquement) :

- **identity** : comptes, rôles (agent)(resident)(conseil), invitations (ASP.NET Core Identity, émission JWT)
- **copropriété** : residences, groupes_habitation, immeubles, lots, residents, lot_resident
- **paiement** : paiements, ajustements_solde
- **assemblees** : assemblees
- **maintenance** : signalements, maintenance_planifiee
- **ged** : documents
- **notification** : preferences_notification, devices, notifications_log

Avec un DbContext unique, les **clés étrangères fonctionnent nativement** entre modules (ex. paiements.lot_id → lots.id, mise à jour du solde par jointure/transaction) et les migrations sont centralisées. Chaque module garde toutefois ses **EntityTypeConfiguration<T>** dans son dossier, appliquées au contexte via **ApplyConfigurationsFromAssembly**, pour rester organisé.

Les colonnes `*_path` pointent vers un fichier sur le **filesystem de la VM**, jamais le contenu binaire en base.

8. Stack technique

Backend (.NET 10 LTS / C# 14)

- Monolithe modulaire **ASP.NET Core Web API** (modules : Identity, Copropriété, Paiement, Assemblées, Maintenance & Signalements, GED, Notification)
- **Communication inter-modules** : interfaces injectées (synchrone) + événements internes **MediatR**
- **ORM** : EF Core ; **une base PostgreSQL**, un schéma par module
- **Auth** : ASP.NET Core Identity + JWT
- **Jobs de fond** : Hangfire / Quartz.NET / `BackgroundService` (purge, notifications)
- **Logs** : Serilog
- **Dev local** : .NET Aspire (optionnel) ou Docker Compose

Frontend BO : React.js (Vite) **Mobile** : React Native + Expo ; push via Expo Notifications / FCM-APNs ; test en dev via **Expo Go**

Base de données & stockage

- **PostgreSQL auto-hébergé** en conteneur Docker sur la VM (gratuit, sans plafond)
- **Sauvegarde** : `pg_dump` quotidien (chiffré) → **Cloudflare R2** (hors-VM, gratuit)
- **Fichiers** : filesystem de la VM (volume Docker) + métadonnées en base ; **job de purge** sur les justificatifs uniquement

Notifications

- **Push** (Expo / FCM-APNs) — seul canal au lancement
- **WhatsApp Cloud API** — reporté en v2

Exports BO : Excel (annuaire, soldes, états) ; PDF optionnel (QuestPDF) si besoin

Infra & DevOps

- **Docker + Docker Compose** sur **une seule VM** (Oracle Cloud Always Free 2 OCPU / 12 Go / 200 Go, ou Hetzner ~5 €/mois pour plus de fiabilité)
- **nginx** (reverse proxy) + **Let's Encrypt** (TLS gratuit)
- Web React en build statique sur **Cloudflare Pages** / **Netlify** (gratuit) ou servi par nginx
- **Pas de Kubernetes** au MVP (k3s plus tard si besoin)
- **CI/CD** : **GitHub Actions** (2 000 min/mois gratuites sur dépôts privés ; illimité sur dépôts publics)

9. Exigences non fonctionnelles

- **Sécurité / accès** : isolation des données par **polices d'autorisation** centralisées dans l'API (rôle + appartenance, à partir des claims JWT) ; un résident n'accède qu'à ses lots, paiements et signalements ; un agent qu'à ses résidences.
- **Confidentialité (loi 09-08)** : consentement explicite pour les canaux de communication ; données personnelles minimisées et protégées.
- **Traçabilité** : journalisation (Serilog) des validations de paiement (qui, quand) et des envois de notifications.
- **Fichiers** : justificatifs et documents sur le filesystem (volume privé) + métadonnées en base ; limite de taille (~5-10 Mo).
- **Rétention & purge** : les **justificatifs de paiement** sont purgés automatiquement après un délai (~3 mois — à *valider selon les obligations légales de conservation comptable*) ; les **documents GED** (règlement, PV) sont **conservés indéfiniment**.
- **Sauvegarde** : `pg_dump` quotidien chiffré poussé hors-VM (Cloudflare R2).
- **Multi-résidence natif** : la structure résidence → GH → immeuble → lot fonctionne identiquement pour 1 ou N immeubles.
- **Évolutivité** : le module paiement accueillera CMI en v2 (mode auto-validé) ; un module peut être extrait en service si la charge l'exige.

10. Hypothèses et points ouverts

- **Solde par lot** (et non par résident) ; l'app mobile présente l'agrégat au résident.
- **Alimentation du solde** assurée au MVP par la saisie d'ajustements/charges par l'agent (pas d'appels de fonds automatisés).
- **Délai de rétention des justificatifs** (~3 mois) : à valider juridiquement avant mise en place de la purge.
- **Conseil syndical** : rôle prévu mais non prioritaire en v1.
- **Émargement AG** : à inclure ou non en v1 selon l'effort.
- **Modification des coordonnées par le résident** : libre ou soumise à validation de l'agent.